UNIVERSITY OF OKLAHOMA

GRADUATE COLLEGE

THESIS TITLE

A THESIS

SUBMITTED TO THE GRADUATE FACULTY

in partial fulfillment of the requirements for the

Degree of

MASTER OF SCIENCE

By SPENCER SMITH

Norman, Oklahoma

2026

THESIS TITLE

A THESIS APPROVED FOR THE
SCHOOL OF COMPUTER SCIENCE

BY THE COMMITTEE CONSISTING OF

Dr. Richard Veras, Chair

Dr. Jie Cao

Dr. Anindya Maiti

© Copyright by SPENCER SMITH 2026

All Rights Reserved.

Acknowledgments

The computing for this project was performed at the OU Supercomputing Center for Education & Research (OSCER) at the University of Oklahoma (OU).

This material is based upon work supported by the National Science Foundation under Grant No. 2440646.

Table of Contents

Acknowledgments iv

List of Figures viii

List of Tables x

Abstract xi

1 Introduction **1**

2 Background **2**

2.0.1 Sparse Matrix Storage Formats 2

2.0.2 SpGEMM 3

2.0.3 BLIS 3

2.0.4 Roofline Model 4

2.0.5 TACO 4

2.0.6 Kronecker Graphs 4

2.0.7 Optimization 5

3 Literature Review **6**

4 Repetition Test Library **7**

4.1 Motivation 8

4.2 Example Usage 9

5 PACT Stuff, title this better. **14**

5.1 Introduction . . . 14

5.2 Mechanism / Methodology . . . 16

5.2.1 Instrumentation and Benchmarking . . . 16

5.2.2 Formula Derivations . . . 19

5.2.3 Format Selection Optimization . . . 22

5.2.4 Sparse with BLIS structure . . . 24

5.2.5 Kronecker Graphs . . . 26

5.3 Analysis . . . 28

5.3.1 Format Benchmarking and Instrumentation Results . . . 28

5.3.2 Format Selection Optimization Results . . . 30

5.3.2.1 Blocking Sensitivity . . . 31

5.3.2.2 Thread Sensitivity . . . 32

5.3.3 Kronecker Graph Results . . . 32

5.4 Summary . . . 32

6 Recent stuff, title this better. **45**

6.1 Introduction . . . 45

6.2 Results . . . 47

7 FPGA Stuff, title this better. **49**

7.1 Theory . . . 49

7.2 Mechanism . . . 50

7.2.1 Implementation . . . 50

7.3 Analysis . . . 56

7.3.1 Preliminary Results . . . 56

7.3.2 Future Work . . . 57

7.4 Summary 58

8 Discussion and Future Work **60**

A Additional Figures **64**

A.1 Results 64

List of Figures

5.1 Observed and Formula FLOPs. Observed data points fall mostly on the dotted line plotted from formulas. 30
5.2 Observed and Formula memops. Observed data points fall mostly on the dotted line plotted from formulas. 34
5.3 FLOPs: Relative error of Formula vs Observed. Very close fit by the formula, and flop count is determined entirely by compression combination. 35
5.4 memops: Relative error of Formula vs Observed. Close fit by most formulas, variability due to the effect of variability of sparsity patterns. 36
5.5 Runtime Performance: Timestamp Cycles. Crossover points at 0.7% density when performing a simple dense x dense is better. 37
5.6 Roofline Model. Sparse formats are entirely memory bound. 38
5.7 Diagonal Matrix Results. Poor performance compared to simple global compression, but optimal micro-format selection wins over naive format selection. 39
5.8 Block Size Performance Impact. Performance degrades as block sizes increase. 40
5.9 Thread Count Performance Impact. At matrix sizes that can be solved, threading provides minimal benefit beyond 2 threads. 41

5.10 Kronecker Graph Sweep (AS-Newman). Sparse BLIS performs very poorly compared to a global compression strategy, this worsens as matrix size increases. 42
5.11 Kronecker Graph Sweep (AS-Newman). Sparse BLIS performs very poorly compared to a global compression strategy, this worsens as matrix size increases. 43
5.12 Kronecker Graph k=10 (AS-Newman) block size sweep. Block dispatch overhead generally decreases as block size increases, but not nearly enough to be comparable to simple global compression. 44

6.1 TODO: This needs to be broken up, this is unreadable, probably do the grid based? or perhaps just try letting each sub-figure be its own figure while still having each kronecker to be on the same one... more comparable that way... 47
6.2 TODO: this one is going to be humongous with all of them. Also need to do this across more than just varying K, again don't want to make the mistake of confounding the difference come from structure when it may just be due to density or size... 48

7.1 Current Sequential Implementation 50
7.2 Conceptual View of Matrix Stream 52
7.3 Future Parallel Implementation 58

List of Tables

5.1 Arithmetic and memory operation count formulae for all sixteen format combinations. matches $= L_{NZ} \cdot R_{NZ}/L_{CC}$, $L_S = \min(L_{RC}, L_{NZ})$, $R_S = \min(L_{CC}, R_{NZ})$. 22

7.1 Stream Memory Layout: fixed-size header followed by format-specific data. 53

7.2 Memory Usage for Matrix Size 57

7.3 FPGA Synthesis Resource Usage 57

Abstract

Put your abstract here.

Chapter 1

Introduction

> TODO

Chapter 2

Background

> TODO: This is the background section from PACT paper... need look over and revise.

2.0.1 Sparse Matrix Storage Formats

The four formats considered in this work represent the standard vocabulary of sparse matrix storage. Compressed Sparse Row (CSR) stores non-zeros row by row, with a row pointer array enabling efficient traversal of row entries. Compressed Sparse Column (CSC) is the column-major analog: columns are stored contiguously, making it efficient for column-wise access patterns. Coordinate format (COO) stores each non-zero as an explicit $(\text{row}, \text{col}, \text{value})$ triple, meaning pointer compression but consuming more memory per entry and imposing a scan cost proportional to the total non-zero count during row-wise access. Dense format stores all elements explicitly in a flat row-major array, eliminating indexing overhead entirely and exposing maximum memory locality, making it optimal when most entries are non-zero, unsurprisingly. Each format encodes the same mathematical object but makes different trade-offs between storage overhead, access pattern regularity, and the cost of skipping structural

zeros. The standard reference for these formats in the context of sparse linear algebra compilers is [1], who provide a unified description of format capabilities in terms of per-dimension storage modes.

2.0.2 SpGEMM

SpGEMM computes $C = A \times B$ where A and B are sparse matrices. Unlike sparse matrix-vector multiplication (SpMV), SpGEMM involves two sparse operands whose non-zero structure interacts during multiplication: the output sparsity pattern is not known in advance and must be computed symbolically before numeric accumulation can begin. The operation appears as a core subroutine in algebraic multigrid solvers, triangle counting, multi-source breadth-first search, and high-dimensional similarity search, among many other applications [2]. Buluç and Gilbert [3] provide foundational parallel algorithms for SpGEMM. The most performance-relevant observation for this work is that SpGEMM is overwhelmingly memory-bound on current hardware: the irregular, format-dependent access patterns of sparse operands generate large numbers of short, unpredictable memory requests, keeping arithmetic units underutilized.

2.0.3 BLIS

The BLAS-like Library Instantiation Software (BLIS) framework [4] provides a portable infrastructure for implementing dense level-3 BLAS operations. Its key insight is that virtually all computation in dense matrix multiplication can be expressed in terms of a small, hardware-specific micro-kernel operating on dense register-sized panels, surrounded by a fixed five-loop blocking structure parameterized by cache-level constants m_c, n_c, k_c and micro-kernel block sizes m_r and n_r. This separation allows the bulk of the framework to be written once in portable C while performance-critical sections

are tuned per architecture. Our implementation preserves this five-loop structure and blocking hierarchy, replacing only the innermost micro-kernel with a dispatch over heterogeneous sparse formats.

2.0.4 Roofline Model

The roofline model [5] provides an upper bound on achievable kernel performance as a function of arithmetic intensity (FLOPs per byte of memory traffic). For a given kernel and machine, the model draws a "roofline" as the minimum of the peak compute throughput and the product of memory bandwidth and arithmetic intensity. A kernel is compute-bound if its arithmetic intensity exceeds the machine's compute-to-bandwidth ratio, and memory-bound otherwise. SpGEMM kernels consistently fall in the memory-bound regime due to low data reuse and irregular access patterns, a finding confirmed by our results in Section 4.

2.0.5 TACO

The Tensor Algebra Compiler (TACO) [6] automatically generates efficient kernels for any compound tensor algebra expression over mixed dense and sparse tensors, including all format combinations relevant to this work. TACO provides a principled formalization of how iteration spaces and index expressions vary across storage formats, and its generated kernels served as the starting point and correctness reference for our hand-instrumented implementations.

2.0.6 Kronecker Graphs

Kronecker graphs [7] are a family of self-similar synthetic networks generated by repeated Kronecker product of a small initiator matrix with itself. They were introduced

as a model for real-world networks because their recursive structure naturally produces the heavy-tailed degree distributions and small diameters observed in internet topology graphs and social networks. The AS-Newman autonomous systems dataset used in our experiments is a standard benchmark graph commonly represented as a Kronecker graph instance.

2.0.7 Optimization

Format selection in this work is formulated as a Mixed Integer Nonlinear Program (MINLP) and solved using Pyomo [8] with MindtPy [9] as the MINLP solver backend, using GLPK for MIP subproblems and IPOPT for the NLP relaxations.

Chapter 3

Literature Review

Chapter 4

Repetition Test Library

This chapter discusses the library I created for benchmarking which was used in most experiments excepting any experiments utilizing FPGAs. Though not all functionality was present through all experiments using it. It was inspired and initially based architecturally on the one covered by Casey Muratori's Computer Enhance project [10]. It builds off of and requires my own base layer library of C code. It, like most of my C code, currently only supports Linux, but especially this particular library because it relies on the Linux 'Perf' [11] syscall API for collecting CPU hardware counters.

As one might assume from its title, it is primarily used to repeatedly test functions, collecting info about them such as timing, page faults, in addition to the aforementioned hardware counters for things such as branch and cache misses. In my own biased opinion, it provides this functionality in a straightforward and flexible way. It will continue to collect this info in a loop, collecting new minimums to try to eliminate any OS-scheduling or other noise. It allows programs to own their own control flow, while simply marking up important sections for benchmarking and data collection. It also allows users to provide additional info to calculate second order statistics such as

throughput by providing byte counts. Finally, it allows easy access to this data via options to output as CSV, containing both library generated data and user-provided data. It also pushes live info as it is collected to the terminal.

4.1 Motivation

The experiments in this thesis required a library/harness with a particular combination of capabilities which none of the options I considered provided all of.

Hardware counters for diagnosis. The experiments in Chapter 5 compare operation counts against runtime, and the gap between the two cannot be explained by operation counts alone nor do they necessarily even predict real performance. At this point the library did not collect hardware counter events, but explaining performance in reality requires cache misses, branch mispredictions, and other hardware events. Collecting these through source-level instrumentation, as I did with the counting macros in that chapter, perturbs the performance being measured, in addition to being tedious to insert. Hardware counters read via the Linux `perf` interface [11] avoid this, so a harness that exposed them directly was necessary.

Measurement of marked regions, not whole processes. Tools such as `perf stat` report counters for entire processes, which includes setup, input generation, verification, etc. I wanted measurements scoped to individual kernels, so the harness had to let the program and user mark the regions of interest.

Program-owned control flow. Many benchmarking frameworks invert control. Often they require the user to register a function and the framework or library decides when and how to call it. This is awkward for kernels that need per-experiment setup,

such as matrix generation at a chosen density, or that are driven from a sweep loop in the host program, or that have special blocking transformations that need to be done on input matrices. I wanted the program to own its own control flow and simply mark the section to be repeated and measured.

Simplicity. My code is written in C on top of a small base library of my own. I considered heavier weight libraries like PAPI [12], which would have potentially allowed me to collect even more cross-microarchitecture (TODO: better way to say 'cross-microarchitecture'?) hardware counter events such as flop/load/store instructions retired easier. However, I decided against it in the interest of keeping benchmarking complexity low. I wanted to save most of my mental complexity budget for dealing with the actual experiments. As well, its always more interesting to build your own thing than use a library.

4.2 Example Usage

Below I provide a simple example program demonstrating the library's use. Note that the user is free do define the order and sections of interest. The example usage below is not the only way to use the library, but it has the general structure most of my experiments ended up in.

> Probably this giant ass listing needs to go in the appendix...

Listing 4.1: Example usage of Repetition Test Library

```
// NOTE: Define the interface that all the functions we want to test conform
    to.
typedef void (*Test_Function)(u8 *buffer, usize buffer_size);
```

```
static
void read_buffer_linearly(u8 *buffer, usize buffer_size)
{
  ... // Omitted for brevity
}
static
void read_buffer_randomly(u8 *buffer, usize buffer_size)
{
  ... // Omitted for brevity
}

// NOTE: Each entry is just going to be the function and
// then a little string to name it.
typedef struct Function_Entry Function_Entry;
struct Function_Entry
{
  const char *name;
  Test_Function function;
};
// NOTE: Ok now we can just make an array of all the functions we want to
    test.
Function_Entry entries[] =
{
  {"linearly", read_buffer_linearly},
  {"randomly", read_buffer_randomly},
};
```

```
int main(int argc, char **argv)
{
  Arena arena = arena_make(.reserve_size = GB(64));
  u64 cpu_timer_frequency = estimate_cpu_timer_freq();

  // NOTE: How many seconds to keep trying for a new min, will reset once
  // we find a new minimum.
  u32 seconds_to_try_for_min = 1;

  // NOTE: User may provide custom fields.
  Repetition_Series *series = repetition_series_make(60 * STATIC_COUNT(
      entries), ((const char *[]){"buffer_size", "function"}));

  for (usize buffer_size = MB(1); buffer_size <= MB(4); buffer_size *= 4)
  {
    for (usize test_func_idx = 0; test_func_idx < STATIC_COUNT(entries);
        test_func_idx++)
    {
      // NOTE: Allocating and deallocating every time to make it fair page-
          fault wise.
      u8 *buffer = os_allocate(buffer_size, OS_ALLOCATION_COMMIT);

      Function_Entry *entry = &entries[test_func_idx];

      Repetition_Tester tester = repetition_series_new_tester(series,
          buffer_size,
                                                              cpu_timer_frequency,
```

```
                                                    seconds_to_try_for_min
                                                        ,
                                                    "---␣%s␣@␣%lu␣MB␣---
                                                        ", entry->name,
                                                    buffer_size / MB(1))
                                                        ;

    // NOTE: User may set fields of current test.
    repetition_series_set_field(series, "buffer_size", "%lu", buffer_size);
    repetition_series_set_field(series, "function", entry->name);

    // NOTE: The main loop, it will continue testing until the minimum time
        has not decreased in user-provided time-frame.
    while (repetition_series_is_testing(series, &tester))
    {
      repetition_tester_begin_time(&tester);
      entry->function(buffer, buffer_size);
      repetition_tester_close_time(&tester);

      repetition_tester_count_bytes(&tester, buffer_size);
    }

    os_deallocate(buffer, buffer_size);
  }
}

// NOTE: Self-explanatory
```

```
  repetition_series_save_csv(series, "out.csv");
}
```

Chapter 5

PACT Stuff, title this better.

> Need to clean this up for thesis... This might need to be split up a bit... Big listing is a little ugly...

5.1 Introduction

SpGEMM is a important kernel in graph analytics, scientific computing, and machine learning, and its performance can depend heavily on how the input matrices are stored. Dense, CSR, CSC, and COO each exploit different sparsity patterns. Usually a single format is chosen to compress an entire matrix. However, real-world matrices often have varying sparsity patterns across the whole matrix, e.g. dense diagonal bands surrounded by very sparse blocks, like those often seen in kronecker graphs. This chapter explores one avenue of exploiting this structured sparsity by partitioning each input matrix into smaller blocks and storing each block in whichever format would suit it best for the operation.

This chapter will investigate whether that idea paid off. The hypothesis was that assigning storage formats per block with a custom cost model and executing the

multiply operation within a BLIS-inspired block loop structure would outperform a single global format. I went about this in this way:

1. I implemented SpGEMM kernels for all sixteen pairings of the four formats, instrumented them to count floating point and memory operations, and derived analytical cost formulae for each.

2. I formulated per-block format selection as a mixed integer nonlinear program using those cost formulae, and built a sparse BLIS pipeline that executes the resulting heterogeneous assignment.

3. I evaluated the pipeline against uniform-format and global-compression baselines on synthetic banded matrices and finally on Kronecker graphs.

The results as you will see are mixed. The cost formulae predict operation counts quite accurately for 14/16 of the format combos, and some benchmarking reveals some crossover points between formats across varying density. Brining it all together in the heterogenous BLIS format did not succeed, however. There is too much overhead from both the dense blocking structure and from format dispatch. Even when the solver chose more 'optimal' formats it did not ever beat simple global compression. Operation counts also proved as a poor proxy for runtime. And finally, the solver does not scale beyond small matrices and proved intractable at sizes where blocking could potentially have benefits.

I believe that even where this experiment failed is still informative so I present all the results in full.

Attribution. This chapter is based on work conducted with Dr. Richard Veras and Moez Durrani. I implemented the kernels, instrumentation, solver pipeline, and

experiments, and performed the analysis and wrote this chapter. Dr. Richard Veras proposed the initial research direction and provided supervision. Moez Durrani provided the kronecker datasets used in some of the experiments.

5.2 Mechanism / Methodology

5.2.1 Instrumentation and Benchmarking

Each matrix multiplication kernel was initially derived using the TACO compiler as a reference, then rewritten by hand to incorporate a custom instrumentation layer. Each kernel is itself actually implemented as a C preprocessor macro, allowing the same implementation to be invoked both within the benchmarking framework and directly as a callable function in the sparse BLIS multiplication pipeline. The instrumentation layer is then built on top of this using a second set of macros, `LOAD`, `STORE`, and `FMADD`, which in their counting mode expand to the underlying operation plus a bit of bookkeeping to record memory and floating point operation activity. Each `FMADD` increments a floating point operation counter by two, while each `LOAD` and `STORE` increments both a memory operation counter and a byte counter. In their benchmarking mode the macros expand to only the base operations with no book-keeping overhead, allowing accurate runtime measurement. We observed the bookkeeping had a measurable and noticeable negative impact on performance.

Runtime performance is measured using a repetition testing framework inspired by Casey Muratori's Computer Enhance material [10]. For each kernel, the framework repeatedly executes the multiplication over a fixed time window and records the minimum observed cycle count, using the x86 hardware timestamp counter, _RDTSC. Taking the minimum rather than the mean reduces the effect of OS scheduling noise

Listing 5.1: Instrumentation macros

```
#ifdef OBSERVE_FLOPS
#define FMADD(dst, a, b) dst += (a * b); \
  repetition_tester_count_flops(tester, 2)
#endif // OBSERVE_FLOPS
#ifdef OBSERVE_MEMOPS
#define LOAD(src) src; \
  repetition_tester_count_memops(tester, 1); \
  repetition_tester_count_bytes(tester, sizeof(src))
#define STORE(dst, src) dst = src; \
  repetition_tester_count_memops(tester, 1); \
  repetition_tester_count_bytes(tester, sizeof(dst))
#endif // OBSERVE_MEMOPS
```

as well as cold cache effects.

By sweeping input matrices across a range of densities, both the counters and the runtime measurements can be collected. Of course one at a time to eliminate any noise from the instrumentation on performance. We then obtain enough information for a direct comparison between predicted and observed operation counts as a function of sparsity. This sweep also provides the data required to plot a roofline model. The roofline performance bounds of peak floating point throughput and peak memory bandwidth were measured directly by running hand-written assembly loops designed to fully saturate the FMA and memory load ports of the target CPU. All results are visualized using a Python script built using Matplotlib [13].

The instrumentation macros actually ended up serving a dual purpose during development: beyond the data collection, their presence as visual markers in the kernel source made it straightforward to reason about which operations contributed to each term in the cost formula derivations described in the following subsection.

Listing 5.2: Implementation macro example, backslashes ommitted for formatting, STATEMENT expands to a do while(0)

```
#define csr_x_csr_impl STATEMENT(
  for (usize left_row = 0; left_row < left.row_count; left_row++)
  {
    usize left_row_start = LOAD(left.row_pointers[left_row]);
    usize left_row_end = LOAD(left.row_pointers[left_row + 1]);

    for (usize i = left_row_start; i < left_row_end; i++)
    {
      usize left_col = LOAD(left.col_indices[i]);
      f64 left_value = LOAD(left.values[i]);

      usize right_row_start = LOAD(right.row_pointers[left_col]);
      usize right_row_end = LOAD(right.row_pointers[left_col + 1]);
      for (usize j = right_row_start; j < right_row_end; j++)
      {
        usize right_col = LOAD(right.col_indices[j]);
        f64 right_value = LOAD(right.values[j]);

        usize output_index = left_row * output.col_count + right_col;
        f64 current_value = LOAD(output.values[output_index]);

        f64 result_value = current_value;
        FMADD(result_value, left_value, right_value);

        STORE(output.values[output_index], result_value);
      }
    }
  }
)
```

5.2.2 Formula Derivations

To enable format selection without runtime measurement, we derive analytical formulae for the floating point and memory operation counts of each kernel as a function of matrix dimensions and sparsity. For each kernel, the derivation follows a consistent procedure: we walk the loop nest from outermost to innermost, identify every `LOAD`, `STORE`, and `FMADD` site, and multiply each by the number of times its enclosing loop body executes. The instrumentation macros serve as explicit markers during this process, making each counted operation visually identifiable in the source code. We also note that we assume a uniform sparsity.

In some cases the loop bounds are not immediately obvious. For instance a loop iterating from a sparse row pointer start to end appears to iterate over a single row, but summing across all rows reveals the total iterations equal the non-zero count of that matrix. Recognising these collapsed bounds where nested loops together amount to a single pass over all non-zeros was an important step in several of the derivations.

We adopt a more descriptive subscript notation in place of the conventional M, N, K.

We define the following notation throughout:

- L_{RC}, L_{CC} — row and column count of the left matrix
- R_{CC} — column count of the right matrix (inner dimension is shared: $L_{CC} = R_{RC}$)
- L_{NZ}, R_{NZ} — non-zero counts of the left and right matrices
- matches $= L_{NZ} \cdot R_{NZ} / L_{CC}$ — expected number of multiply-accumulate operations under uniform sparsity with 2 sparse formats

Representative Derivations

We will present a few full derivations which we find representative of the whole process.

Dense $\times$ CSR The outer loop iterates over L_{RC} rows of the left matrix. Within each row, the middle loop iterates over all L_{CC} rows of the right CSR matrix, reading two row pointers per iteration, $2 \cdot L_{RC} \cdot L_{CC}$ reads in total. However, across all iterations of this middle loop, the inner loop collectively visits every non-zero of the right matrix exactly once, so the total 2 inner iterations collapse to R_{NZ} per output row. This gives one left value load per non-zero per row, contributing another $L_{RC} \cdot L_{CC}$, and four reads/writes per matched non-zero the right column index, right value, output load, and output store giving $4 \cdot L_{RC} \cdot R_{NZ}$.

$$\text{flops} = 2 \cdot L_{RC} \cdot R_{NZ} \tag{5.1}$$

$$\text{memops} = 3 \cdot L_{RC} \cdot L_{CC} + 4 \cdot L_{RC} \cdot R_{NZ} \tag{5.2}$$

COO $\times$ COO The outer loop scans `left.row_indices` in full to group left entries into row segments, contributing L_{NZ} reads. Within each segment, the merge intersection advances `left.col_indices` through the segment, across all segments this totals another L_{NZ} reads. The right row index array resets to zero at the start of each left row segment and is fully scanned each time, costing $R_{NZ} \cdot L_S$ reads where L_S is the number of distinct left row segments. This is bounded above by both L_{RC}, the total number of rows, and L_{NZ}, since you cannot have more distinct rows than non-zeros, finally giving $L_S = \min(L_{RC}, L_{NZ})$. The min operation here helps to account for how this variable will change as density increases. At low densities L_{NZ} is very small and most non-zeroes exist on different rows, this flips as density increases and

we approach a case where each row has at least one non-zero. Of course this holds true only for uniform sparsity. On a match, `left.values` is loaded once per matched left entry, giving L_{NZ} reads, and the right column index, right value, output load, and output store contribute $4 \cdot \text{matches}$. The expected match count assumes uniform sparsity, where each left non-zero at column k expects R_{NZ}/L_{CC} right non-zeros with row k, giving matches $= L_{NZ} \cdot R_{NZ}/L_{CC}$.

$$L_S = \min(L_{RC}, L_{NZ}) \tag{5.3}$$

$$\text{matches} = L_{NZ} \cdot R_{NZ}/L_{CC} \tag{5.4}$$

$$\text{flops} = 2 \cdot \text{matches} \tag{5.5}$$

$$\text{memops} = 3L_{NZ} + R_{NZ} \cdot L_S + 4 \cdot \text{matches} \tag{5.6}$$

Two format combinations, CSR×CSC and COO×CSC, present additional derivation difficulty compared to the rest. Both involve a merge intersection where both operands reset on different outer loop dimensions simultaneously: in CSR×CSC the left row entries reset per output column and the right column entries reset per output row, while in COO×CSC the same interaction occurs but compounded by the left row segment structure common in multiplies with COO. This seems to produce terms in the memory cost that are difficult, at least for us, to derive. The formulae given in Table 5.1 for these two cases capture the observed structure to the best of our abilities, though with considerably more error than the rest. All other fourteen combinations show strong agreement with observed counts across the full density range, as discussed in the Analysis section.

Left	Right	Flops	Memops
DNS	DNS	$2L_{RC} \cdot L_{CC}R_{CC}$	$2L_{RC}L_{CC}R_{CC} + L_{RC}R_{CC}$
DNS	CSR	$2L_{RC} \cdot R_{NZ}$	$3L_{RC}L_{CC} + 4L_{RC}R_{NZ}$
DNS	CSC	$2L_{RC} \cdot R_{NZ}$	$3L_{RC}R_{CC} + 3L_{RC}R_{NZ}$
DNS	COO	$2L_{RC} \cdot R_{NZ}$	$2R_{NZ} + L_{RC}R_{NZ} + 4L_{RC}R_{NZ}$
CSR	DNS	$2L_{NZ} \cdot R_{CC}$	$2L_{RC} + 2L_{NZ} + 3L_{NZ}R_{CC}$
CSR	CSR	$2 \cdot$ matches	$2L_{RC} + 4L_{NZ} + 4 \cdot$ matches
CSR	CSC	$2 \cdot$ matches	$2L_{RC} + 4L_{RC}R_{CC} + L_{NZ}R_{CC} + L_{RC}R_{NZ} + 2 \cdot$ matches
CSR	COO	$2 \cdot$ matches	$2L_{RC} + L_{NZ} + L_{RC}R_{NZ} + 4 \cdot$ matches
CSC	DNS	$2L_{NZ} \cdot R_{CC}$	$2L_{CC} + 2L_{NZ} + 3L_{NZ}R_{CC}$
CSC	CSR	$2 \cdot$ matches	$4L_{CC} + 2L_{NZ} + 4 \cdot$ matches
CSC	CSC	$2 \cdot$ matches	$2R_{CC} + 4R_{NZ} + 4 \cdot$ matches
CSC	COO	$2 \cdot$ matches	$2R_{NZ} + 2R_S + 4 \cdot$ matches
COO	DNS	$2L_{NZ} \cdot R_{CC}$	$3L_{NZ} + 3L_{NZ}R_{CC}$
COO	CSR	$2 \cdot$ matches	$5L_{NZ} + 4 \cdot$ matches
COO	CSC	$2 \cdot$ matches	$L_{NZ} + 2R_{CC}L_S + L_{NZ}R_{CC} + R_{NZ}L_S + 2 \cdot$ matches $+$ L_SR_{CC}
COO	COO	$2 \cdot$ matches	$3L_{NZ} + R_{NZ}L_S + 4 \cdot$ matches

Table 5.1: Arithmetic and memory operation count formulae for all sixteen format combinations. matches $= L_{NZ} \cdot R_{NZ}/L_{CC}$, $L_S = \min(L_{RC}, L_{NZ})$, $R_S = \min(L_{CC}, R_{NZ})$.

5.2.3 Format Selection Optimization

Given the cost formulae derived in the previous subsection, we formulate the problem of selecting an optimal storage format for each micro-block as an integer optimization program. Since in matrix multiplication one block may actually be multiplied against many other blocks. The objective is to minimise the total estimated cost across all block multiplications required to compute the full matrix product

Each input matrix A and B is partitioned into micro-blocks of fixed size $L_{RC} \times L_{CC}$ and $L_{CC} \times R_{CC}$ respectively. The dimensions can also be viewed as those usually found at the M_r and N_r size found in BLIS. The density of each block is computed by simply counting its non-zeros, and put into one of our density buckets. A cost table is then computed for all combinations of left format, right format, and density bucket pair using the above derived formulae, treating flops and memory operations

as a combined cost weighted by an assumed memory access penalty.

For each block of A at position (i, p) and each block of B at position (p, j), we introduce binary indicator variables $a_{i,p,f}$ and $b_{p,j,f}$ denoting whether block (i, p) of A or block (p, j) of B is assigned format $f \in \{\text{dense, CSR, CSC, COO}\}$. A single-assignment constraint ensures each block is assigned exactly one format:

$$\sum_{f} a_{i,p,f} = 1 \quad \forall\, i, p \tag{5.7}$$

$$\sum_{f} b_{p,j,f} = 1 \quad \forall\, p, j \tag{5.8}$$

The objective minimises the total estimated cost over all block multiplications:

$$C = \min \sum_{i,j,p} \sum_{f_A, f_B} \text{cost}[f_A, f_B, d^A_{i,p}, d^B_{p,j}] \cdot a_{i,p,f_A} \cdot b_{p,j,f_B} \tag{5.9}$$

The product $a_{i,p,f_A} \cdot b_{p,j,f_B}$ makes this objective 'bilinear' in the decision variables, which constitutes a Mixed Integer Nonlinear Program (MINLP). The solver is implemented using Pyomo [8] with MindtPy [9] as the MINLP solver backend, using GLPK [14] for the MIP subproblems and IPOPT [15] for the NLP solver. Once solved, the optimal format assignments are written alongside the input matrices to a binary file consumed by the sparse BLIS multiplication program.

- $B_{row} = \texttt{MATRIX_SIZE}/L_{RC}$ — number of block rows in A/'Left'
- $B_{col} = \texttt{MATRIX_SIZE}/R_{CC}$ — number of block columns in B/'Right'
- $B_{inner} = \texttt{MATRIX_SIZE}/L_{CC}$ — number of blocks along the shared inner dimension

So far, the primary limitation of this approach is solver scalability. The number of decision variables grows as $O(B_{row} \cdot B_{inner} \cdot B_{col} \cdot |Formats|)$ where $|Formats| = 4$ is the number of formats, and the bilinear objective compounds this as problem size increases. In practice the solver becomes intractable for matrix sizes beyond approximately 512×512 with the micro-block sizes used in our experiments. One direction may be increasing the micro-block size to alleviate the rapid growth of the number of decision variables.

5.2.4 Sparse with BLIS structure

The sparse BLIS implementation follows the hierarchical loop structure of BLIS [4], preserving its cache-level blocking parameters m_c, n_c, k_c and micro-kernel blocking parameters m_r, n_r, k_u as compile-time constants. Rather than operating on scalar elements or fixed-format compressed matrices, the innermost micro-kernel operates on heterogeneous sparse blocks — each of size $m_r \times k_u$ for the left matrix and $k_u \times n_r$ for the right — where each block may independently be stored in any of the four formats.

Prior to multiplication, each input matrix is packed into a `Multisparse_Matrix` structure by the `multi_sparsify` function, which partitions the dense input into blocks and converts each block to its assigned format according to the format assignment produced by the ILP solver, in the order consumed by the multiply. This packing is performed once and before the actual operation begins, as opposed to the standard BLIS practice where packing into the A_c and B_c panels happens during operation.

The multiplication itself is structured as the standard five-loop BLIS nest, but now iterating over block indices rather than element indices. At the innermost level, each

Listing 5.3: Multisparse Matrix struct

```
struct Matrix_Union
{
  Matrix_Format format;
  union
  {
    Dense_Matrix dense;
    CSR_Matrix csr;
    CSC_Matrix csc;
    COO_Matrix coo;
  };
};
struct Multisparse_Matrix
{
  // Overall.
  usize row_count;
  usize col_count;

  Matrix_Union *blocks;
  usize blocks_row_count;
  usize blocks_col_count;
};
```

pair of left and right blocks is dispatched to the appropriate kernel via a computed enum value encoding the format combination, covering all sixteen format pairings. The result of each micro-kernel is accumulated into a small dense temporary buffer of size $m_r \times n_r$ before being written back to the output matrix, very similarly to the original BLIS.

Three configurations are benchmarked against each other: the solved optimal heterogeneous format assignment, a uniform block format baseline where all micro-blocks use the same user-specified format, and a global compression baseline where the entire matrix is compressed into a single format without any type of block structure. Correctness of the sparse BLIS output is verified by comparison against a reference multiplication within an epsilon.

5.2.5 Kronecker Graphs

Kronecker graphs were selected as an experimental dataset for two reasons. First, their recursive self-similar structure produces adjacency matrices with a hierarchical sparsity pattern, the same block density distribution repeats at every scale which we hypothesized might align naturally with the block-level format assignment of our approach, potentially allowing denser sub-blocks to be stored in dense or CSR format while sparser sub-blocks use COO. Second, the recursive structure of the cost model meant that format assignments computed at a smaller scale might be reusable at larger scales, potentially reducing solver cost. Experiments were conducted using the AS-Newman autonomous systems graph, generated as a Kronecker graph with varying scale parameter k, giving matrices of size $2^k \times 2^k$. We swept k from 7 to 15, running both the Sparse BLIS configuration and the global CSR×CSR baseline on each, with results reported in Section 4.3. In practice however, the self-similar

Listing 5.4: Sparse BLIS Implementation

```
for (usize block_j_o = 0; block_j_o < block_n; block_j_o += BLOCK_J)
{
  for (usize block_p_o = 0; block_p_o < block_k; block_p_o += BLOCK_P)
  {
    for (usize block_i_o = 0; block_i_o < block_m; block_i_o += BLOCK_I)
    {
      for (usize block_j_i = 0; block_j_i < BLOCK_J; block_j_i += 1)
      {
        for (usize block_i_i = 0; block_i_i < BLOCK_I; block_i_i += 1)
        {
          Dense_Matrix temp =
          {
            .values = (f64[BLOCK_MR * BLOCK_NR]) {0},
            .row_count = BLOCK_MR,
            .col_count = BLOCK_NR,
          };
          for (usize block_p_i = 0; block_p_i < BLOCK_P; block_p_i += 1)
          {
            usize block_i = block_i_o + block_i_i;
            usize block_j = block_j_o + block_j_i;
            usize block_p = block_p_o + block_p_i;

            Matrix_Union left_block = left.blocks[block_i
                                    * left.blocks_col_count + block_p];
            Matrix_Union right_block = right.blocks[block_p
                                    * right.blocks_col_count + block_j];
            do_sparse_matmul(temp, left_block, right_block);
          }
          for (usize j_r = 0; j_r < BLOCK_NR; j_r += 1)
          {
            for (usize i_r = 0; i_r < BLOCK_MR; i_r += 1)
            {
              usize row = (block_i_o + block_i_i) * BLOCK_MR + i_r;
              usize col = (block_j_o + block_j_i) * BLOCK_NR + j_r;

              usize output_index = row * output->col_count + col;
              usize temp_index = i_r * temp.col_count + j_r;

              output->values[output_index] += temp.values[input_index];
            }
          }
        }
      }
    }
  }
}
```

sparsity of Kronecker graphs meant that all blocks at every scale fell into the same low density bin, rendering format selection trivial and providing no opportunity for heterogeneous assignment to reduce cost over a uniform sparse format. Due to solver scalability limitations discussed in Section 3.3, matrices larger than 256×256 were run using a constant block format assignment of CSC x CSR rather than the solver. This was in practice inconsequential, because for all Kronecker graphs tested the solver assigned CSC x CSR uniformly anyway, as every block fell into the same low density bin and the cost table therefore selected the same format for every block regardless of position.

5.3 Analysis

5.3.1 Format Benchmarking and Instrumentation Results

For the graphs below, we swept both left and right matrices through a range of densities.

Figures 5.1 and 5.2 show observed and predicted FLOP and memory operation counts across the full density range for all sixteen format combinations, with both left and right matrices swept independently. The derived formulae show strong agreement with observed counts for fourteen of the sixteen combinations across the full density range, with relative error remaining below 5% for most format pairs at densities above 0.1, as shown in Figures 5.3 and 5.4.

The two exceptions, CSR×CSC and COO×CSC, exhibit higher error at low densities. Both involve a merge intersection where both operands reset simultaneously on different outer loop dimensions, producing memory cost terms that resist clean derivation under the uniform sparsity assumption. The formulae for these two cases capture

the qualitative structure but with greater quantitative error than the remainder.

A notable property of the FLOP formulae is that they are exact, that is, floating point operation count is determined purely by the number of matched nonzeros and is independent of format, so all sparse×sparse combinations share the same FLOP formula. The memory operation formulae are where format choice has its effect, encoding the differing access patterns of each combination.

Figure 5.5 shows the runtime performance of all sixteen format combinations across the density sweep. Several crossover points are visible as density increases: sparse formats such as COO and CSR are fastest at low densities where their ability to skip zeros outweighs their indexing overhead, while dense×dense becomes competitive and eventually dominant at higher densities where the cost of sparse bookkeeping exceeds the savings from skipping nonzeros.

Figure 5.6 shows the roofline model for all sixteen combinations. All kernels fall well below the memory bandwidth ceiling and far below the compute ceiling, confirming that these kernels are memory bound at all densities tested. The arithmetic intensity is low across the board, consistent with sparse formats that perform few FLOPs per byte loaded. No kernel approaches the compute roof, indicating that optimizing memory access patterns is more impactful than optimizing arithmetic throughput for this class of operation, unsurprisingly. A notable observation is that CSC×dense performs surprisingly close to dense×dense across the full density range, and was indeed selected by the solver for diagonal blocks in the banded diagonal experiments. This suggests CSC is an unusually cache-friendly format for the left matrix when the right matrix is dense, likely due to its column-major access pattern aligning well with the row-major accumulation into the output.

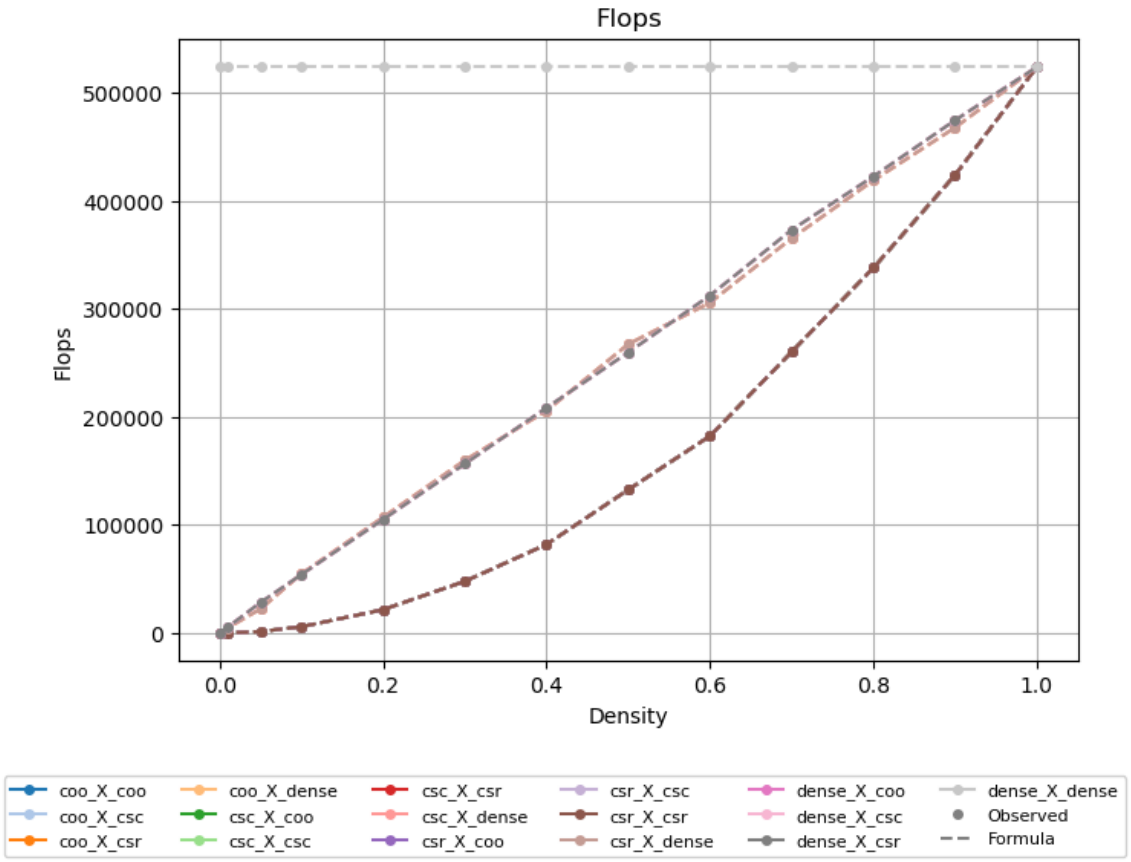


Figure 5.1: Observed and Formula FLOPs. Observed data points fall mostly on the dotted line plotted from formulas.

5.3.2 Format Selection Optimization Results

An important finding concerns the cost model used to drive the format selection. The analytical formulae derived in Section 3.2, while accurate predictors of operation counts, proved to be poor proxies for actual runtime when used directly as the solver objective. Format assignments produced using the formula-based cost model were outperformed by assignments produced using empirically measured cycle counts from the density sweep as the cost function. This suggests that operation count alone is an insufficient proxy for runtime at this block size, likely because cache effects and branch misprediction dominate over raw arithmetic and memory operation counts for

32×32 blocks. In essence, our formulas mistakenly treat flops and memops as being comparable on their face, when they are not. The formulas could perhaps become more predictive at larger block sizes where compute cost dominates over fixed per-block overheads.

Figure 5.7 shows the performance of the three configurations on a synthetic banded diagonal 256×256 matrix, the most favorable case for heterogeneous format selection as it exhibits clear density variation between diagonal and off-diagonal blocks. Even here, the solver-assigned configuration requires 408,725 cycles, compared to 239,189 cycles for global CSR x CSR, a 41% gap. The constant blocking configuration performs worst at 658,040 cycles, suggesting that solver assignment does at least select better formats than a naive uniform choice in this case, but neither blocking approach approaches the performance of the flat global baseline, unfortunately.

Figure 9 shows thread scaling. Performance does not improve meaningfully with additional threads and degrades at 8 threads, indicating the workload is too small and too irregular to benefit from parallelism at this scale.

Figure 10 shows the Kronecker graph sweep. Sparse BLIS scales poorly with matrix size while global CSR×CSR remains effectively flat, reflecting that block dispatch overhead grows with the number of blocks while the actual sparse compute remains minimal due to the uniformly low density of Kronecker graph matrices.

5.3.2.1 Blocking Sensitivity

Figure 5.8 shows performance across block sizes of 32, 64, and 128. Cycle count increases with block size, likely due to increased cache pressure from the growing dense accumulation buffer and greater work per block pair in the inner loop nest.

5.3.2.2 Thread Sensitivity

Figure 5.9 shows thread scaling. Performance does not improve meaningfully with additional threads and degrades at 8 threads, indicating the workload is too small and too irregular to benefit from parallelism at this scale.

5.3.3 Kronecker Graph Results

Figure 5.11 shows the Kronecker graph sweep. Sparse BLIS scales poorly with matrix size while global CSRxCSR remains effectively flat, reflecting that block dispatch overhead grows with the number of blocks while the actual sparse compute remains minimal due to the uniformly low density of Kronecker graph matrices.

As noted in Section 3.5, the solver assigned a uniform CSCxCSR format to all blocks for every Kronecker graph tested, collapsing the heterogeneous assignment to a constant blocking scheme identical to the baseline. This confirms that the density variation required to motivate mixed format selection is absent in these datasets.

Figure 5.12 presents a sweep over progressively larger block sizes for Sparse BLIS. While the conventional micro-block dimension is 32×32, this experiment extends into block sizes that can no longer be strictly classified as micro-blocks. Nevertheless, even at these larger granularities, the additional blocking overhead proves insufficient to overcome the inherent inefficiencies, at least for this dataset.

5.4 Summary

We presented a study of SpGEMM across all sixteen pairings of four sparse matrix storage formats, deriving analytical formulas for floating-point and memory operation counts that show strong agreement with empirically measured values for fourteen of

the sixteen combinations.

Building on these formulas, we implemented a BLIS-inspired [4] sparse matrix multiplication pipeline that assigns storage formats independently to each micro-block and dispatches to the corresponding kernel at runtime. Format selection is formulated as a MINLP and solved using Pyomo [8] with MindtPy [9]. Evaluation on banded diagonal matrices and Kronecker graph datasets [7] shows that block dispatch overhead consistently dominates runtime, preventing the heterogeneous approach from matching a flat global CSR×CSR baseline despite producing better format choices than a naive uniform assignment.

Several directions could address these limitations. A more uniform blocking/adaptable format for the inner most microblocks could be explored. A lighter-weight greedy format selection heuristic, rather than a full MINLP solve, could reduce planning overhead. Finally, the approach could be better suited to architectures with less penalty for very branch-y inner loops.

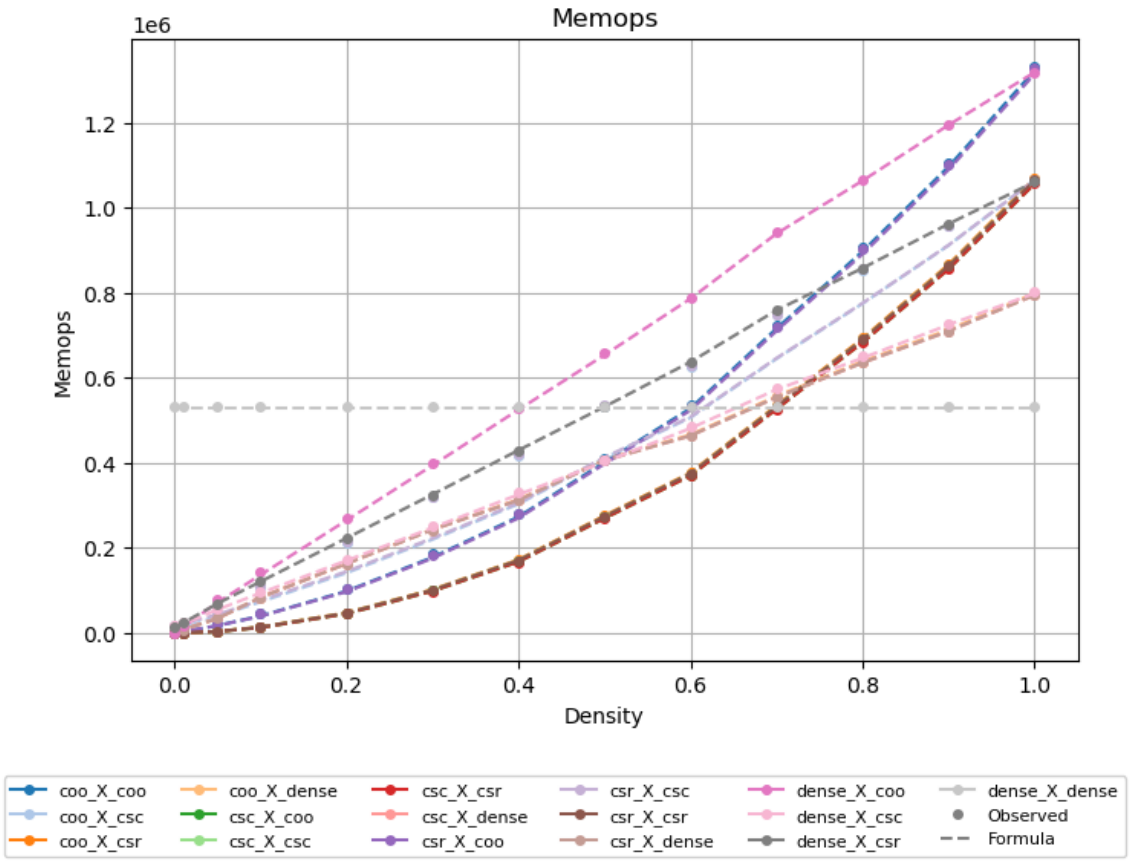


Figure 5.2: Observed and Formula memops. Observed data points fall mostly on the dotted line plotted from formulas.

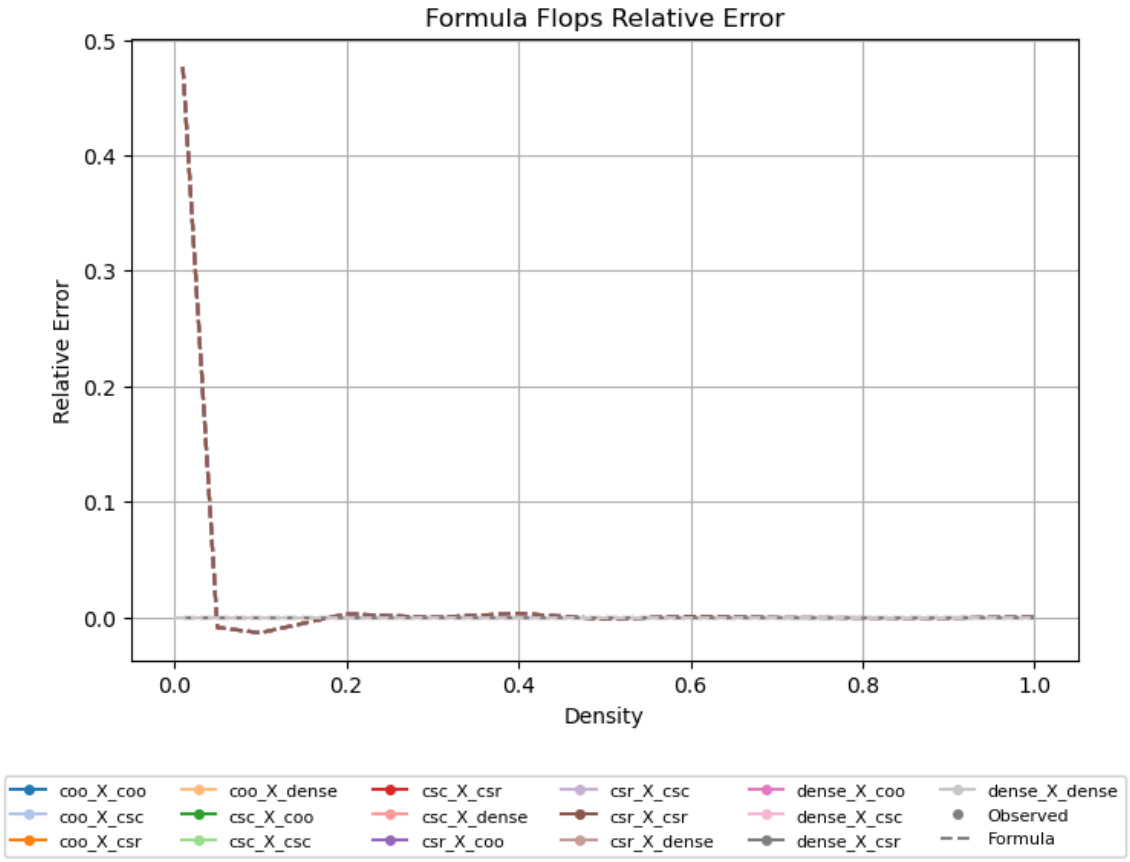


Figure 5.3: FLOPs: Relative error of Formula vs Observed. Very close fit by the formula, and flop count is determined entirely by compression combination.

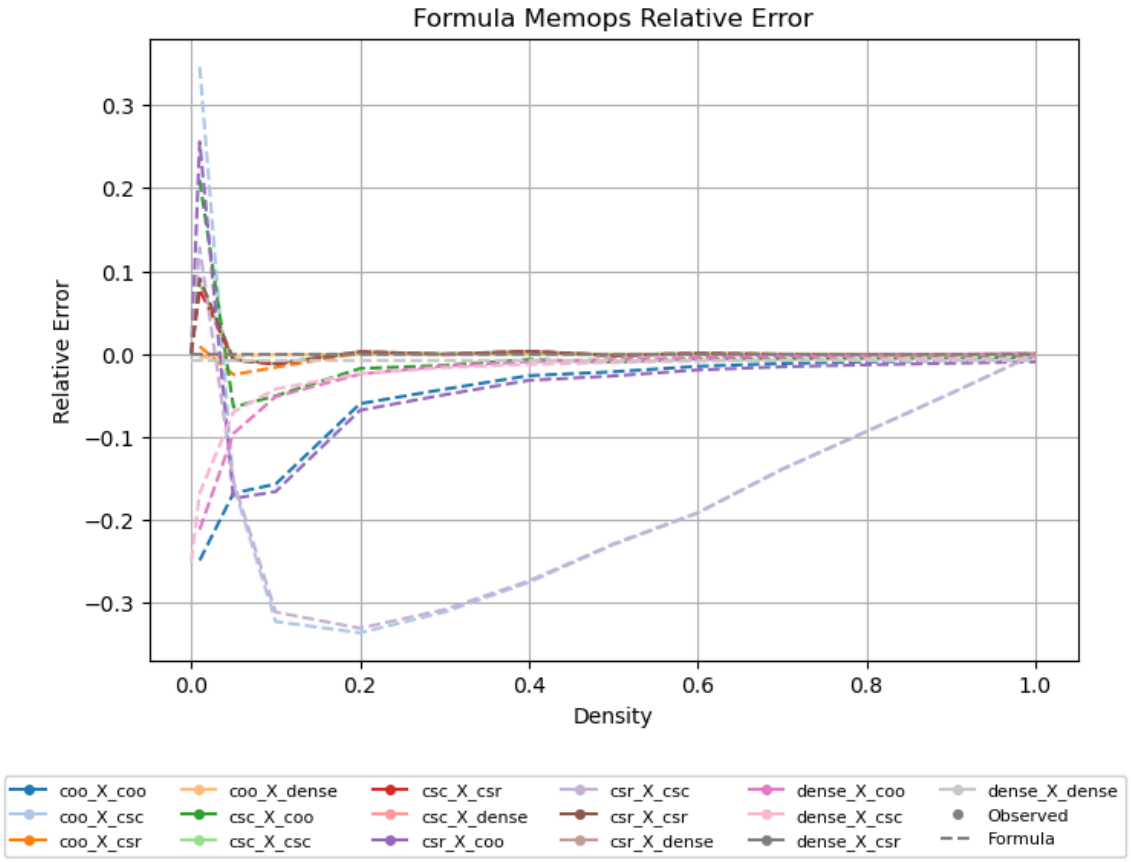


Figure 5.4: memops: Relative error of Formula vs Observed. Close fit by most formulas, variability due to the effect of variability of sparsity patterns.

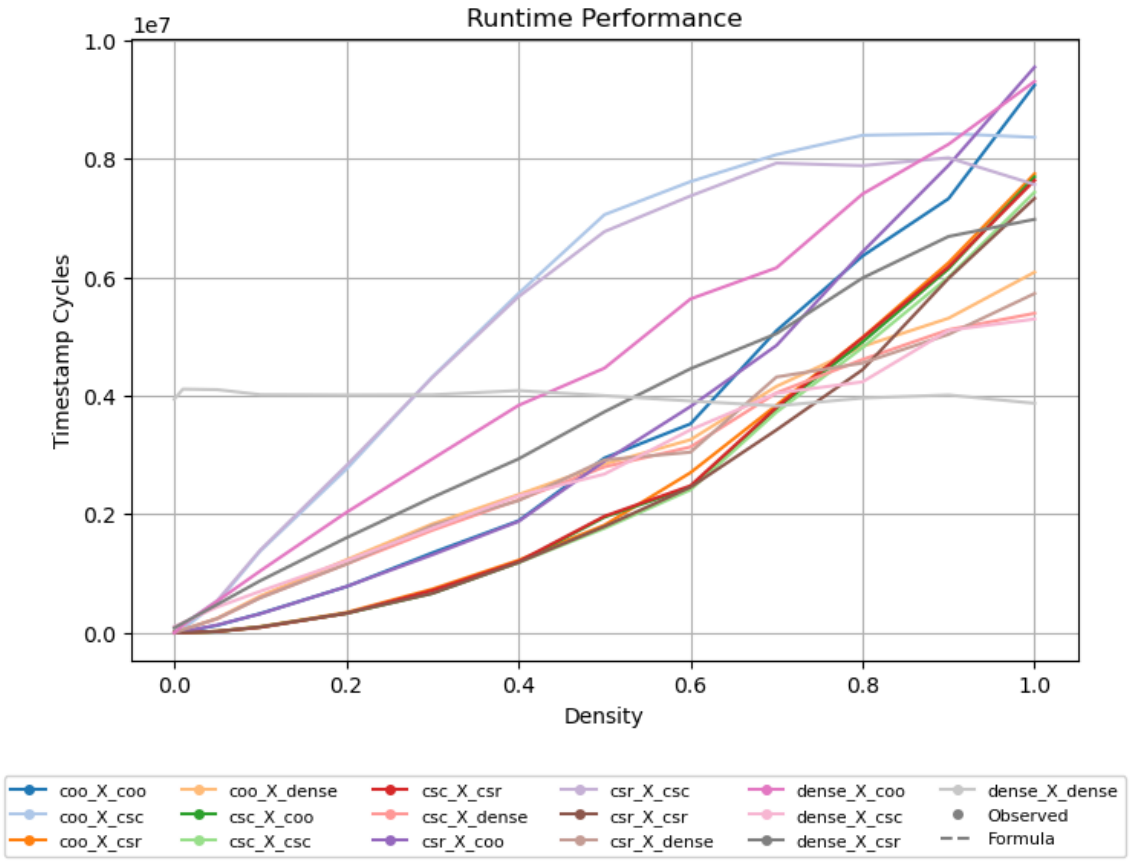


Figure 5.5: Runtime Performance: Timestamp Cycles. Crossover points at 0.7% density when performing a simple dense x dense is better.

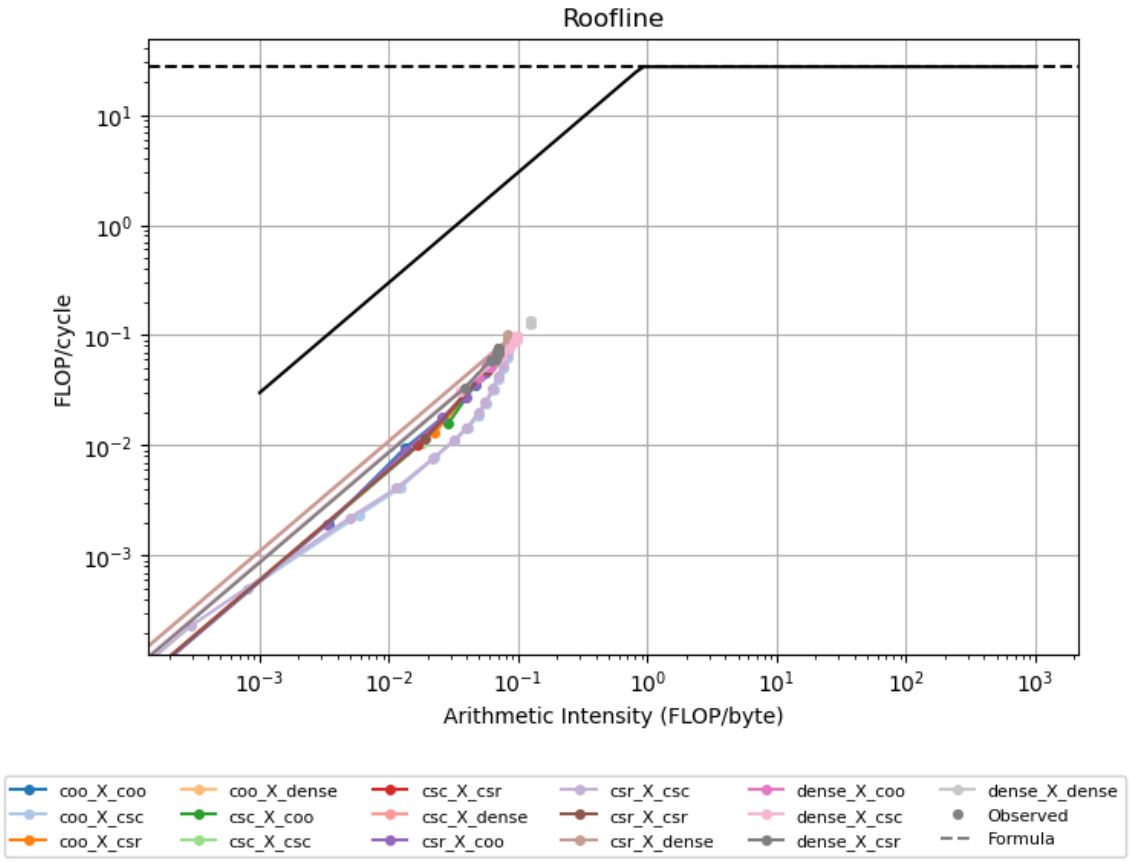


Figure 5.6: Roofline Model. Sparse formats are entirely memory bound.

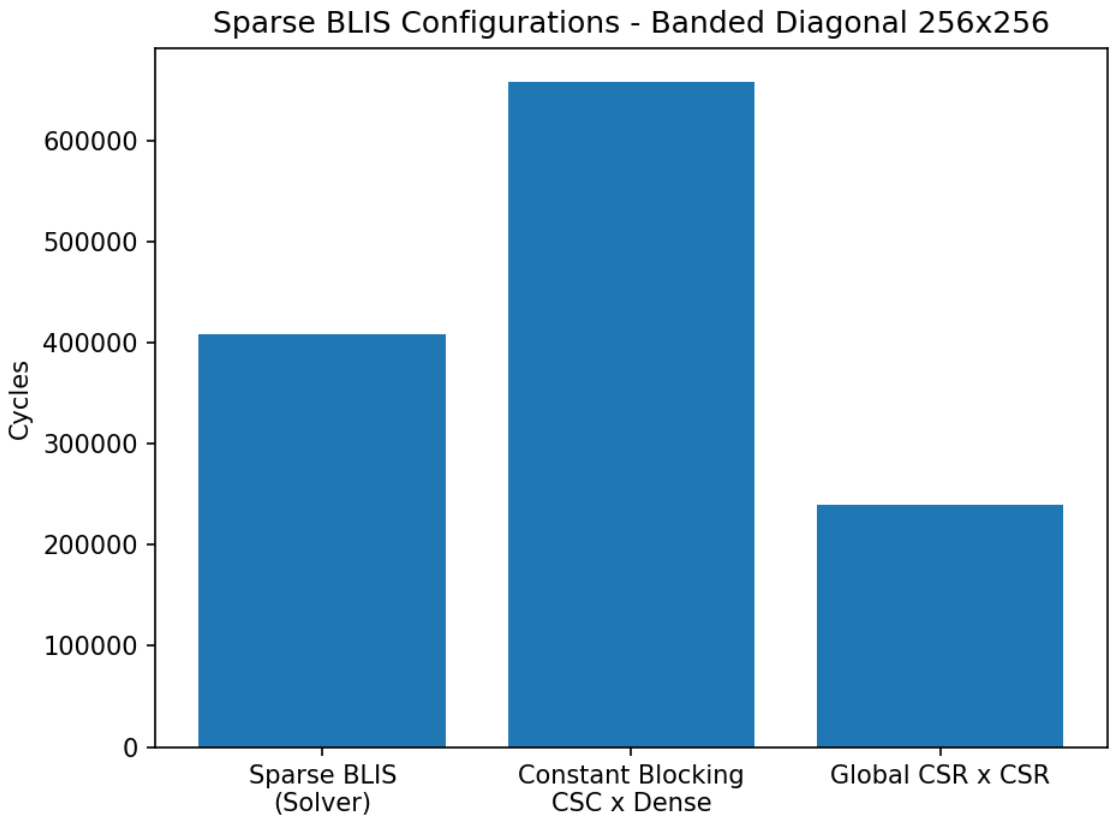


Figure 5.7: Diagonal Matrix Results. Poor performance compared to simple global compression, but optimal micro-format selection wins over naive format selection.

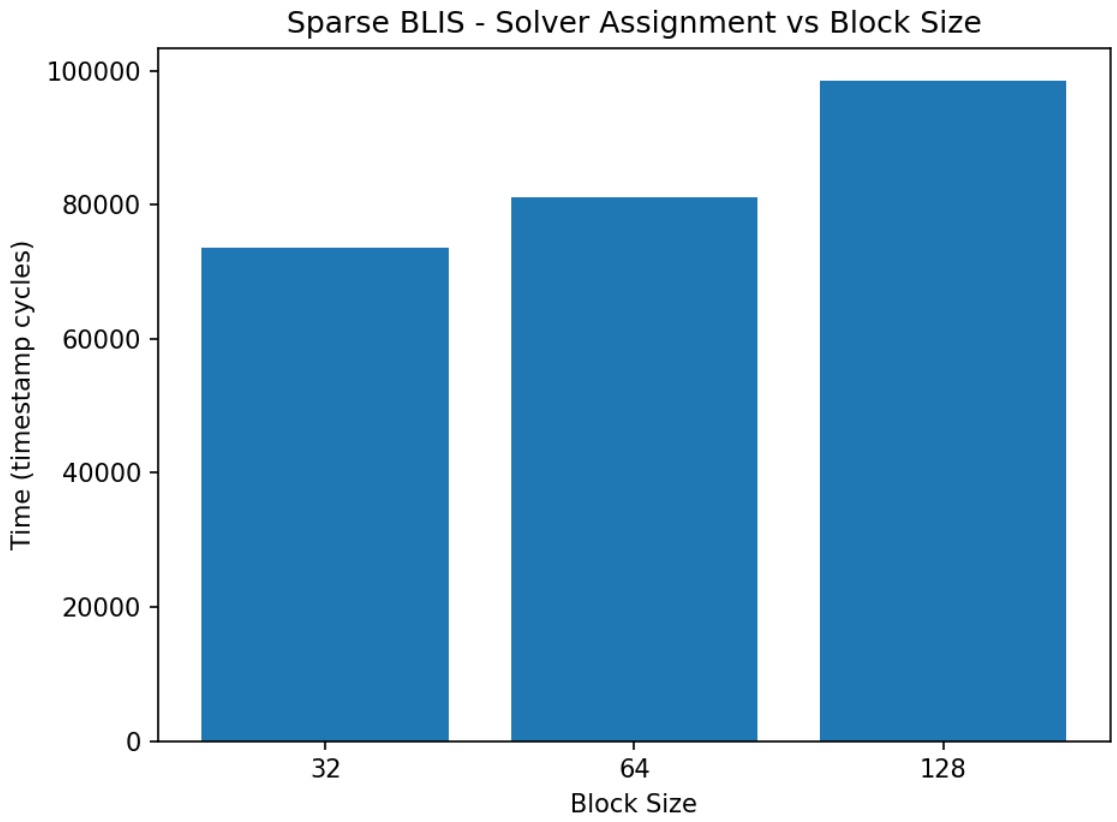


Figure 5.8: Block Size Performance Impact. Performance degrades as block sizes increase.

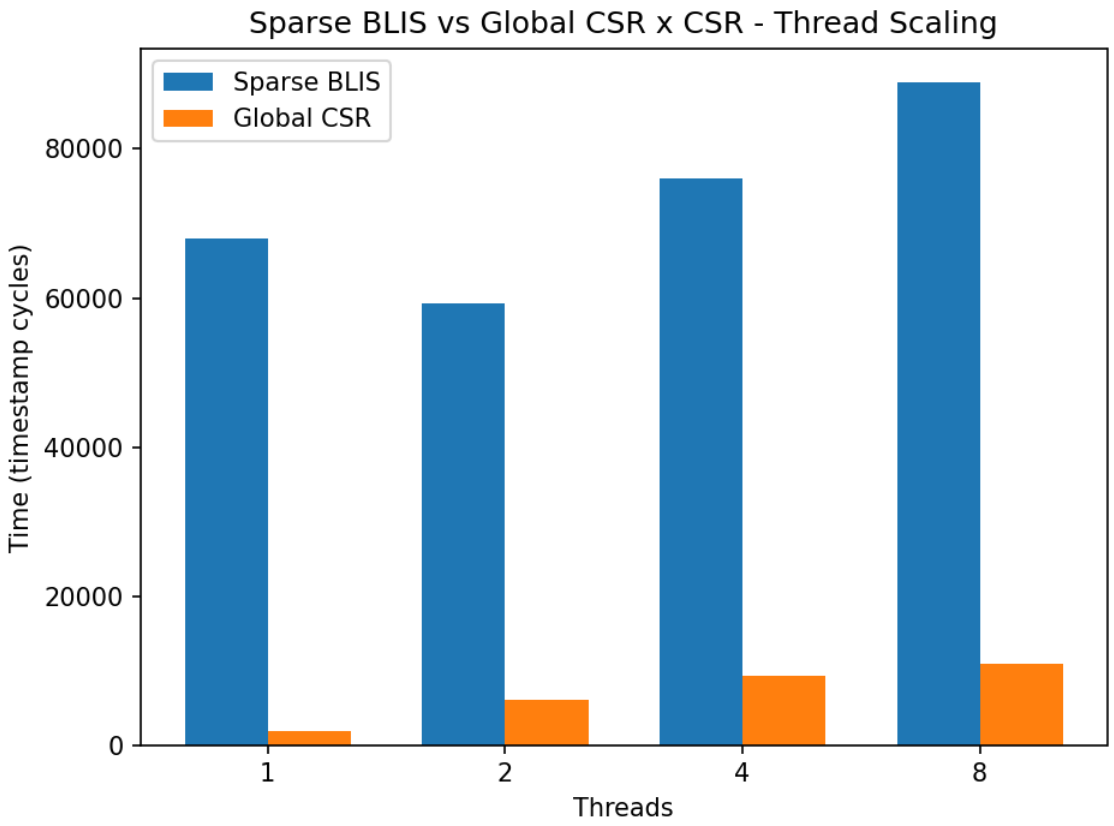


Figure 5.9: Thread Count Performance Impact. At matrix sizes that can be solved, threading provides minimal benefit beyond 2 threads.

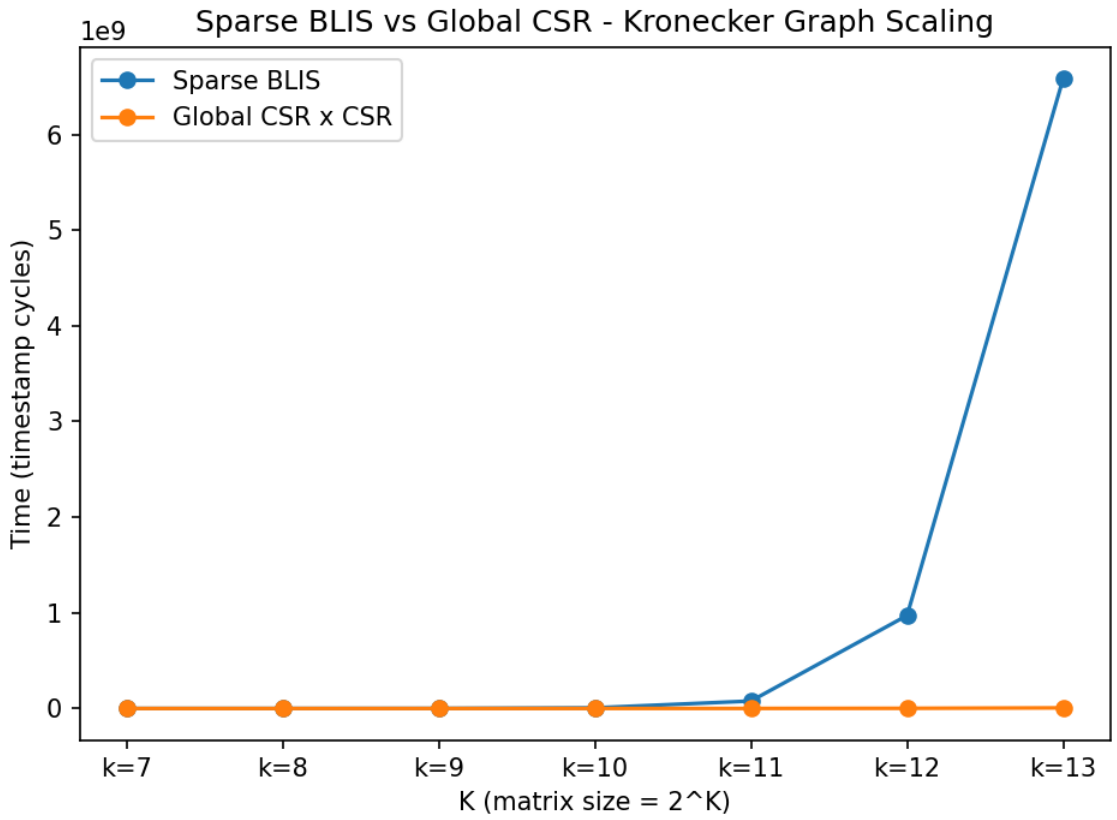


Figure 5.10: Kronecker Graph Sweep (AS-Newman). Sparse BLIS performs very poorly compared to a global compression strategy, this worsens as matrix size increases.

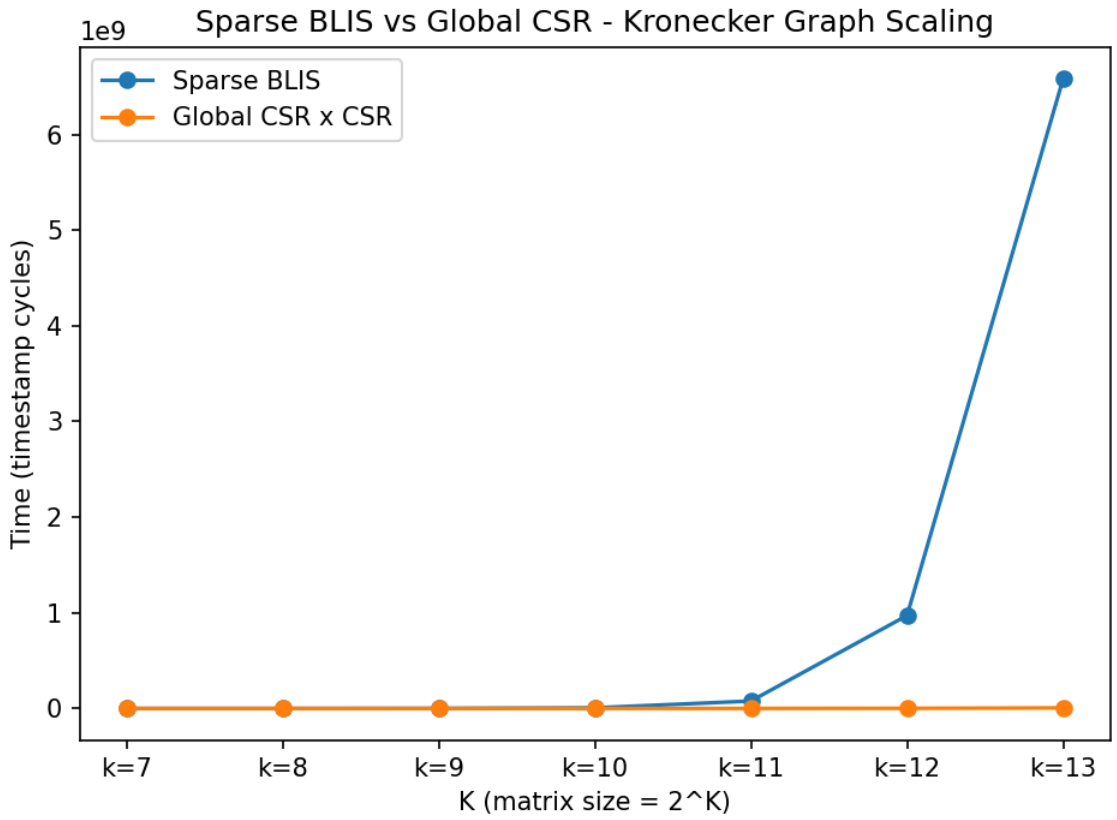


Figure 5.11: Kronecker Graph Sweep (AS-Newman). Sparse BLIS performs very poorly compared to a global compression strategy, this worsens as matrix size increases.

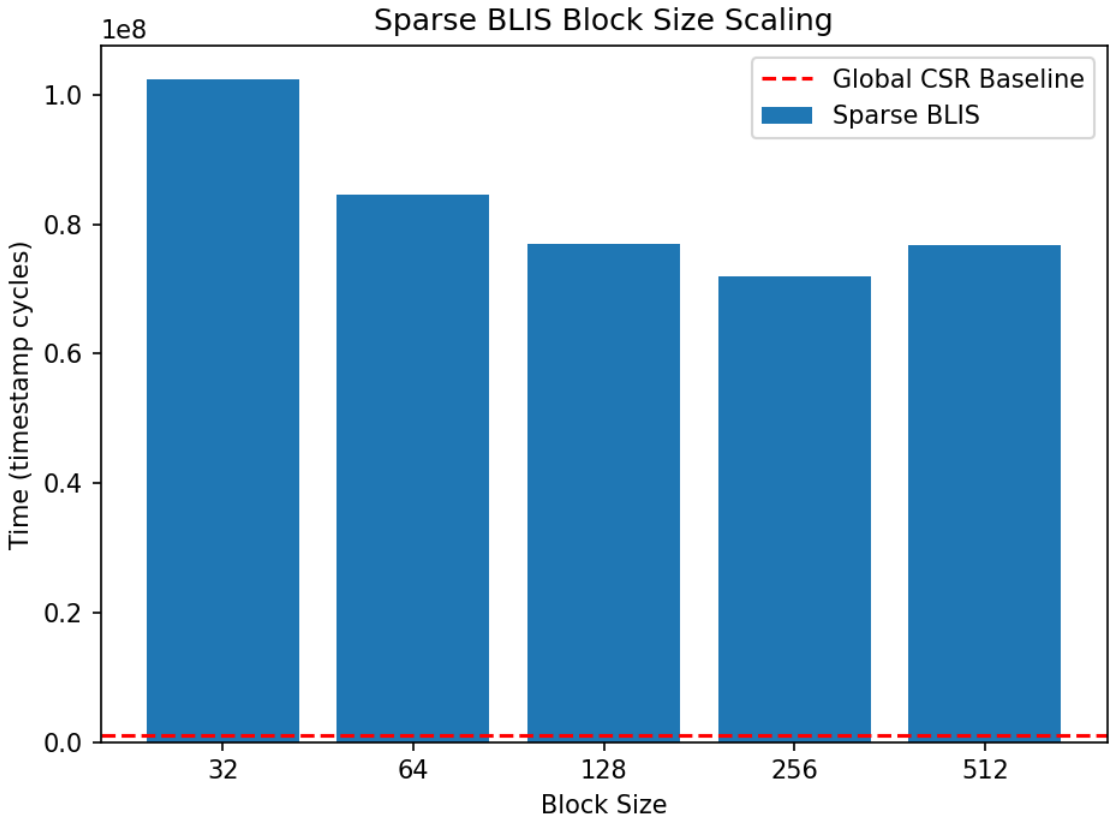


Figure 5.12: Kronecker Graph k=10 (AS-Newman) block size sweep. Block dispatch overhead generally decreases as block size increases, but not nearly enough to be comparable to simple global compression.

Chapter 6

Recent stuff, title this better.

6.1 Introduction

The preceding chapter left me with a few open questions. The analytical cost models of Chapter 5 predicted operation counts accurately, but operation counts proved a very poor proxy for runtime (perhaps predictably, a memory operation in one context is sometimes orders of magnitudes slower than a memory operation in another due to caching), and the heterogeneous per-block pipeline built on them did not outperform a single simple globally chosen format. Those experiments could show that the models failed to predict performance, but not exactly why. They also relied on a small set of synthetic inputs, so they said little about how the best format combination changes as a matrix grows or as its structure changes.

This chapter returns to the these unanswered with the upgraded reptest library and a bit more systematic experiment design. Rather than building a more elaborate selection mechanism first, I asked the more basic question I should have started with. For sparse×sparse matrix multiplication, how does the best pairing of storage formats vary with matrix structure and scale, and, importantly, what hardware behavior

explain these differences? This question informed the design choices I made below.

Structured inputs. Graph adjacency matrices are a common source of sparse matrix inputs, and Kronecker graphs [7] are a natural family to study. They are generated from a small 'initiator' matrix by repeated Kronecker products, so their structure is reproducible while still exhibiting structure commonly associated with real networks. In addition, the same graph family can be generated at many scales.

Sweeping scale instead of comparing named graphs. Sparse linear algebra benchmarks are often reported as results on a collection of well-known, common matrices, typically as one bar per matrix. Such comparisons are informative, but they confound size, density, and structure, and they give no sense of where or why performance behavior changes. Because Kronecker graphs can be generated at successive scales of k, each family can be swept across k, producing continuous curves in which crossover points between formats are visible directly.

Explaining performance with hardware counters. Chapter **??** describes the benchmarking library I built, which reads hardware counters through Linux `perf` sycalls. In the earlier experiments, only operation counts and cycle counts were available, and the gap between them could only be speculated about. In the new experiments, each reptest measurement also records cache misses, branch mispredictions, page faults (which is especially important to reason about for large matrices) so that differences in runtime between format pairings can be tied to specific observed behavior rather than inferred.

Compiler-generated kernels. In Chapter 5, the kernels were rewritten by hand to embed instrumentation. In this chapter, kernels are generated by the TACO compiler

[6], which derives code for any combination of formats from a single algebraic specification with 'scheduling' commands. This makes the comparison between formats more even-handed, since every pairing is produced by the same process. However, it also exposed practical limits of using TACO at scale: generated code assumed 32-bit sizes in allocation, which fails for most kronecker matrices tested of roughly $2^{20} \times 2^{20}$ and above. I describe the changes made to support these sizes in Section **??**.

Scope. I restricted my attention to sparse×sparse products.

> probably more to say here..

Blocking.

> TODO: Once these experiments are finished...

6.2 Taco Limitations

> Oh boy.

6.3 Results

> TODO: Translate notes to academic-speak, also update with most recent figures, also figure out why matplotlib is overlapping the titles...

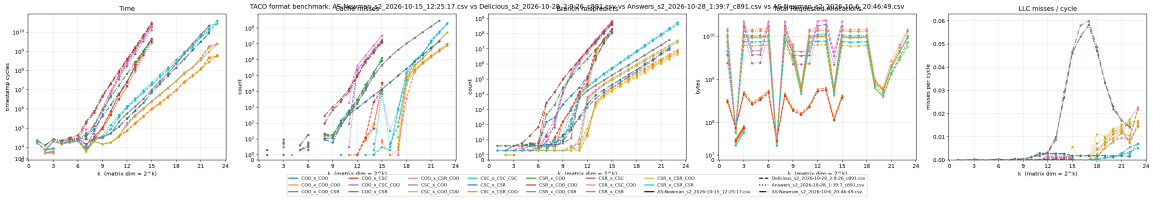


Figure 6.1: TODO: This needs to be broken up, this is unreadable, probably do the grid based? or perhaps just try letting each sub-figure be its own figure while still having each kronecker to be on the same one... more comparable that way...

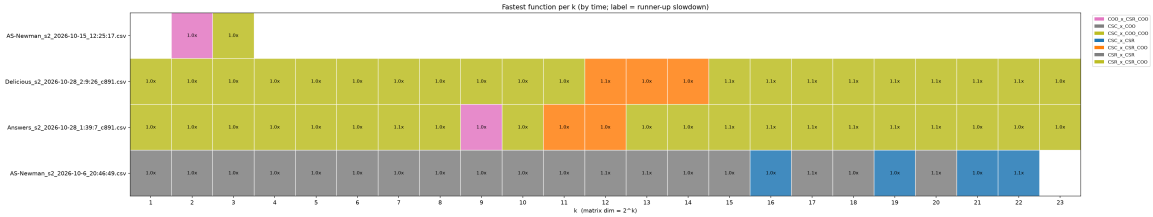


Figure 6.2: TODO: this one is going to be humongous with all of them. Also need to do this across more than just varying K, again don't want to make the mistake of confounding the difference come from structure when it may just be due to density or size... also need to mention that runner-ups are very, very close in almost all cases.

Chapter 7

FPGA Stuff, title this better.

> Need to clean this up for thesis... Need to put refs from here into the .bib

7.1 Theory

To take full advantage of the unique characteristics of highly irregular SpMV operations, this chapter a new streaming multi-format structure. Where single-format matrices may succeed in general or uniform cases, there are others where the sparsity is not uniform. In these cases it may be beneficial to store parts of the larger matrix in the most efficient format. As well, because each value in a matrix for matrix-vector multiplication is read from only once, a streaming data structure may be particularly suited to the task. Furthermore, because of the heterogeneous logic involved in the different algorithms tailored to the different formats, current GPU hardware accelerators may not be sufficient, should any type of parallelism be pursued. Therefore, to address this unique challenge, this chapter proposes implementing this structure on FPGA. In summary, this chapter will discuss a multi-format matrix block stream, in which block sizes may be of arbitrary granularity and of arbitrary format to best

exploit the data layout on a matrix by matrix basis.

7.2 Mechanism

7.2.1 Implementation

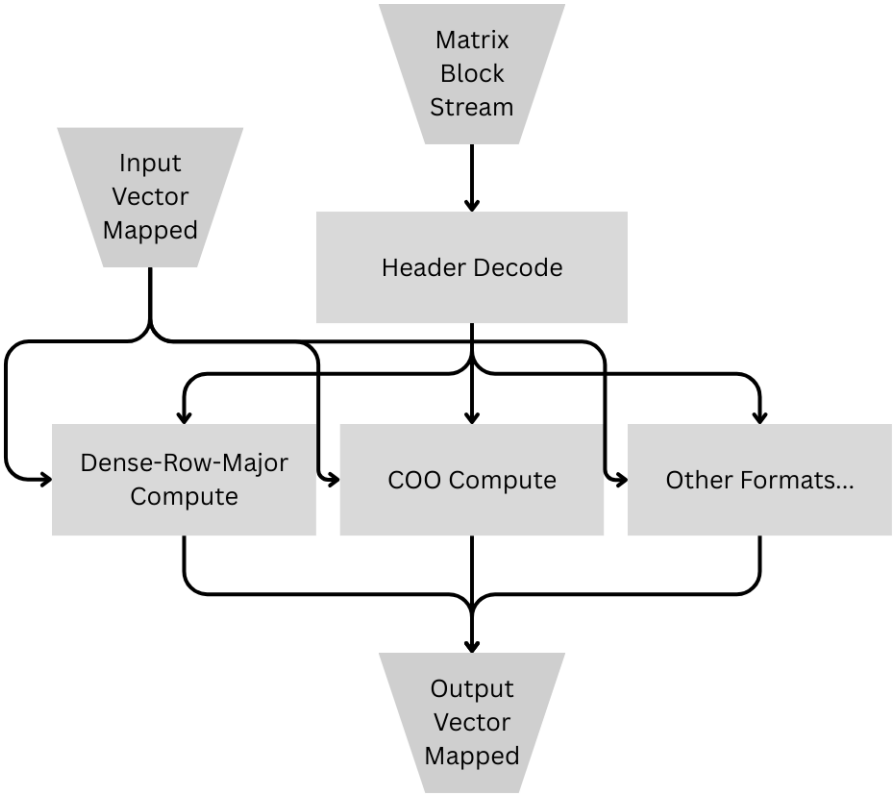


Figure 7.1: Current Sequential Implementation

In this initial exploratory implementation, Figure 7.1, only two simple submatrix format options were chosen: Sparse coordinate (COO) and dense row-major / C-style. These were chosen for the simplicity of streaming. As well [16] found it to be quite resistant to very irregular matrices, meaning a smaller variance. In matrix-vector multiplication, each element of the matrix must only be visited/read from once, making a stream a satisfactory data structure for storing and retrieving matrix

Listing 7.1: Block header structure for mixed-format streaming.

```
struct Mix_Matrix_Header
{
    Matrix_Format format; // Enum: NONE = 0, DENSE = 1, COO = 2
    uint64_t vec_in_offset; // Input vector offset
    uint64_t vec_out_offset; // Output vector offset
    uint64_t vec_count; // Number of rows in the tile
    uint64_t data_size; // Stream offset to next tile
};
```

elements as they are needed. Vitis also provides the hls::stream library for just this purpose. By default, this wraps around a FIFO port when placed on the top interface, so out of order accesses are not allowed [17]. This constraint is not an issue for the two formats chosen. The test bench takes matrices already in COO format sorted by row, then column and divides them into blocks. Each block is tested for a simple threshold of sparsity:

$$\text{Format} = \begin{cases} \text{Dense Row-Major}, & \text{if } 8mn \leq 24nnz, \\ \text{COO}, & \text{otherwise.} \end{cases} \tag{7.1}$$

Here, m and n denote the number of rows and columns in the block, respectively, and nnz is the number of non-zero elements in the block. The constants 8 and 24 represent the number of bytes required to store one element in dense and COO formats, respectively. This heuristic favors dense format when its storage cost is less than or equal to that of COO. Future work may explore more sophisticated format-selection rules.

After format-selection, a header is constructed for the data: Here, each header contains a bit of information about each following block, most importantly the storage format, and the block offsets into the respective input and output vectors. Matrix

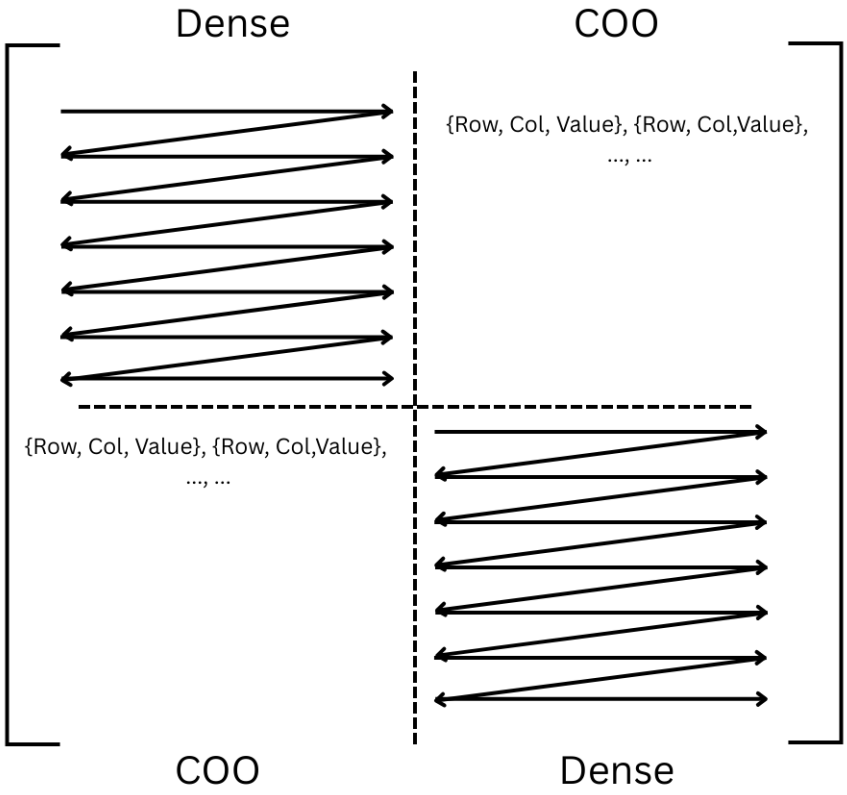


Figure 7.2: Conceptual View of Matrix Stream

data is then written into the stream in the specified format, and in the order the values will be consumed. Values in this implementation are all doubles or 64 bit floats. For the COO format, first the number of non zeroes in the block as a uint64 is streamed, then values are streamed as triples of $(row, column, value)$, exactly the order the COO compute function consumes them. And for Dense, first the number of rows and columns in the block are streamed first, again as uint64's, and then values in row major order. Conceptually, it might be thought of as Figure 7.2.

Table 7.1: Stream Memory Layout: fixed-size header followed by format-specific data.

Bytes	**Field**	**Description**
0–39	Header	Format ID, offsets, vector count
40–(40+N)	Dense Payload	$m \times n$ values
...		
X–(X+39)	Next Header	Same format as first header
...	COO Payload	$nnz \times (row, col, val)$ triples
Y–(Y+39)	NONE Header	Signifies the end of the stream

In reality the stream memory layout is demonstrated by Table 7.1. This stream is then consumed on FPGA, dispatching to the correct format compute function based on the header, Listing 7.2.

One can observe that this could be expanded to any number of potential formats, as long as they were conducive to streaming. The compute function/module for COO is given in Listing 7.3. In hindsight, choosing COO proved to be a little hazardous in the read-modify-write-cycle. Ensuring correctness required disabling pipelining for the COO_LOOP, greatly limiting potential throughput. This revealed a major limitation of some formats that have very irregular access like COO for the streaming, pipe-lined architecture of FPGAs. This as an important lesson guiding future work in exploring other formats. For now these are relatively naive implementations, but one could imagine using the given format's fastest implementation for each compute

Listing 7.2: HLS Stream Processing on FPGA

```
void mix_matrix_mul_vector_top(Data_Stream *mix_stream,
                               f64 *vec_in, u64 vec_in_count,
                               f64 *vec_out, u64 vec_out_count)
{
  ...
  do
  {
    head = read_matrix_header(mix_stream);

    switch (head.format)
    {
      case Matrix_Format::NONE: { } break;
      case Matrix_Format::DENSE_ROW_MAJOR:
      {
        dense_row_major_mul_vec(mix_stream, head,
                                vec_in, vec_in_count,
                                vec_out, vec_out_count);
      } break;
      case Matrix_Format::COO:
      {
        coo_mul_vector(mix_stream, head,
                       vec_in, vec_in_count,
                       vec_out, vec_out_count);
      } break;
    }
  }
  while (head.format != Matrix_Format::NONE);
}
```

Listing 7.3: HLS Stream COO Block Compute

```
void coo_mul_vector(Data_Stream *mix_stream, Mix_Matrix_Header head,
                    f64 *vec_in, u64 vec_in_count,
                    f64 *vec_out, u64 vec_out_count)
{
  u64 non_zero_count = mix_stream->read();

  COO_LOOP:
  for (usize i = 0; i < non_zero_count; i++) {
    u64 coo_row = mix_stream->read();
    u64 coo_col = mix_stream->read();
    f64 coo_value = read_f64_from_stream(mix_stream);

    u64 vec_in_index = coo_col + head.vec_in_offset;
    u64 vec_out_index = coo_row + head.vec_out_offset;

    f64 current = vec_out[vec_out_index];
    f64 vector_in = vec_in[vec_in_index];

    vec_out[vec_out_index] = current + (coo_value * vector_in);
  }
}
```

module, combining the most effective implementation with the most effective format for each blocks data layout. However, this first implementation is very sequential, having only one stream and writing potentially to the same rows in the memory mapped output vector. Both the input and output vectors are AXI Master memory mapped, since we are accessing these values multiple times. Nonetheless we can take advantage of Matrix-Vector multiplication's need to access each matrix value only once with the use of a streaming data structure. The tooling used for this implementation was AMD's Vitis HLS suite [17], and the board we used for our simulations was the Pynq-7000, specifically the xc7z020clg400-1.

7.3 Analysis

7.3.1 Preliminary Results

I created a simple script to generate sparse matrices in edge list format. For this simple exploratory implementation I focused on 2 cases a small 32x32 matrix for initial testing, and a larger matrix of 2048x2048, for a more rigorous test case. In all cases our simple heuristic for deciding format usage achieved a small data footprint, even including stream header overhead, see Table 7.2. However, the simplicity of the rule leaves much to be desired, and further work should be done, both in finding a more efficient rule perhaps even machine learning based, as Chen et al. have [18], as well as expanding the available formats. However, the implementation leaves much of the available hardware unused, as seen by Table 7.3. Ideally, I would like to achieve a much fuller saturation of the DSP units, as well as at least some usage of the fast BRAM. But, there is potential in the streaming structure [19], one could imagine having a much larger sparse matrix that one does not want to materialize

all at once. One could instead only materialize by block, and at that moment also decide the format for streaming. This is especially useful as those blocks can then be dematerialized as they are consumed on FPGA, as stated before matrix-vector multiplication only requires one read for any one value of the matrix.

Table 7.2: Memory Usage for Matrix Size

Matrix Dimension	**Memory Footprint (KB)**
32x32	0.856
2048x2048	9827.224

Table 7.3: FPGA Synthesis Resource Usage

BRAM Usage	**DSP Usage**	**LUT Usage**	**FF Usage**
2%	6%	4%	10%

7.3.2 Future Work

Though there is a working implementation, I believe there is much more potential in this idea. So far, the implementation has not fully utilized the strengths of FPGA as a platform, namely parallelism and BRAM usage. Therefore I propose an improvement to the current design, seen in Figure 7.3. This design would include alongside the other decode and compute modules, a final reduction module. In this implementation, each block and compute module would receive a slice of BRAM to act as a personal subset of the output vector. In this way, each block could run completely in parallel once it received its data from the stream. A final reduce step would sum any overlapping subset output vectors and write back into the memory mapped true output vector. By allowing compute to be pipe-lined more effectively with reading from the stream, we would hopefully achieve much better memory and resource saturation. As mentioned before, slightly more regularly accessed sparse formats would aid

in pipe-lining inner compute loops. In future, I would also like to explore a multiple stream implementation, allowing multiple blocks to be read in parallel.

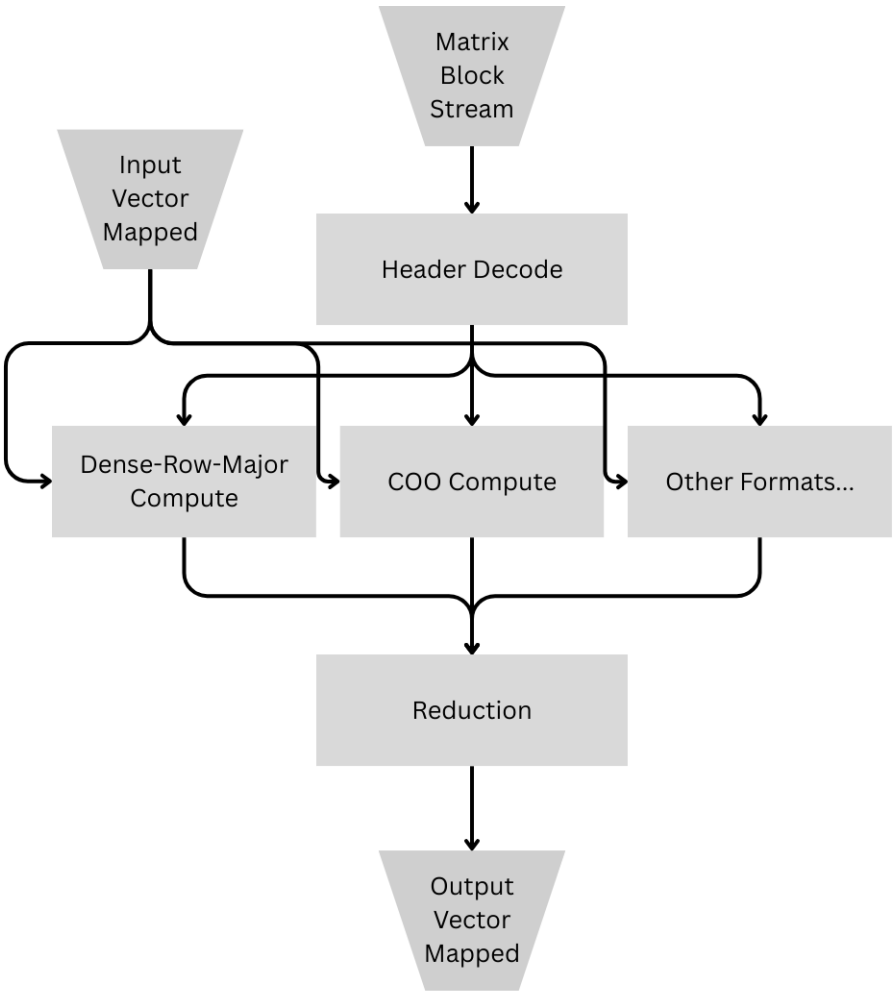


Figure 7.3: Future Parallel Implementation

7.4 Summary

In conclusion, we have motivated the need for new solutions to very irregular sparse matrices, as well as called attention to previous and older solutions. This chapter proposed a novel streaming structure that might combine the strengths of previous

formats, tailoring them to the needed block granularity on an individual matrix basis. In addition, this chapter also describes an implementation of such structure on FPGA to enable future work in parallelizing the heterogeneous computation and unpacking logic of many different formats. Our baseline implementation demonstrated a working computation in which blocks of disparate types were consumed, as well as demonstrated that memory footprint was kept small even with streaming header overhead. Further, this chapter also discussed future work involving usage of BRAM and reduction functional modules for parallelizing computation in hopes of further saturating memory bandwidth.

Chapter 8

Discussion and Future Work

Bibliography

[1] S. Chou, F. Kjolstad, and S. Amarasinghe, "Format abstraction for sparse tensor algebra compilers," in *Proceedings of the ACM on Programming Languages*, vol. 2, 2018. DOI: 10.1145/3276493

[2] J. Gao, Z. Tan, W. Ji, and Y. Zhao, "A systematic survey of general sparse matrix-matrix multiplication," *ACM Computing Surveys*, vol. 55, no. 12, 2023. DOI: 10.1145/3571157

[3] A. Buluç and J. R. Gilbert, "Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments," *SIAM Journal on Scientific Computing*, vol. 34, no. 4, pp. C170–C191, 2012. DOI: 10.1137/110848244

[4] F. G. Van Zee and R. A. van de Geijn, "BLIS: A framework for rapidly instantiating BLAS functionality," *ACM Transactions on Mathematical Software*, vol. 41, no. 3, 2015. DOI: 10.1145/2764454

[5] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009. DOI: 10.1145/1498765.1498785

[6] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe, "The tensor algebra compiler," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, 2017. DOI: 10.1145/3133901

[7] J. Leskovec, D. Chakrabarti, J. Kleinberg, C. Faloutsos, and Z. Ghahramani, "Kronecker graphs: An approach to modeling networks," *Journal of Machine Learning Research*, vol. 11, pp. 985–1042, 2010.

[8] W. E. Hart et al., *Pyomo — Optimization Modeling in Python*, 2nd. Springer, 2017. DOI: 10.1007/978-3-319-58821-6

[9] D. E. Bernal, Q. Chen, F. Gong, and I. E. Grossmann, "Mixed-integer nonlinear decomposition toolbox for Pyomo (MindtPy)," in *Proceedings of the 13th International Symposium on Process Systems Engineering*, ser. Computer Aided Chemical Engineering, vol. 44, 2018, pp. 895–900. DOI: 10.1016/B978-0-444-64241-7.50144-0

[10] C. Muratori, *Performance-aware programming*, Computer, Enhance! (online course), https://www.computerenhance.com, 2023.

[11] Dec. 2024. [Online]. Available: https://perfwiki.github.io/main/

[12] [Online]. Available: https://icl.utk.edu/papi/

[13] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. DOI: 10.1109/MCSE.2007.55

[14] Free Software Foundation, *GNU linear programming kit (GLPK)*, 2023. [Online]. Available: https://www.gnu.org/software/glpk/

[15] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006. DOI: 10.1007/s10107-004-0559-y

[16] G. Flegar and H. Anzt, *Overcoming Load Imbalance for Irregular Sparse Matrices*, Zenodo, Nov. 2017.

[17] AMD Xilinx, *Vitis High-Level Synthesis User Guide (UG1399)*, [Online]. Available, 2023. [Online]. Available: https://docs.amd.com/r/en-US/ug1399-vitis-hls

[18] S. Chen, J. Fang, C. Xu, and Z. Wang, "Adaptive Hybrid Storage Format for Sparse Matrix–Vector Multiplication on Multi-Core SIMD CPUs," *Applied Sciences*, vol. 12, no. 19, p. 9812, 2022.

[19] M. Hosseinabady and J. Nunez-Yanez, "A Streaming Dataflow Engine for Sparse Matrix-Vector Multiplication Using High-Level Synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 6, pp. 1272–1285, Jun. 2020. DOI: 10.1109/tcad.2019.2912923

Appendix A

Additional Figures

A.1 Results